Real-Time Bus Tracking System

Student & Client Presentation Guide — Simple, Plain English Explanation for Teachers & Viva Voce

1. How to Explain This Project in 30 Seconds (The Elevator Pitch)

*"Good morning/afternoon, Teacher. Our project is a **Real-Time Bus Tracking System** built 100% with Java and Spring Boot. It solves the problem of commuters and university students waiting aimlessly at bus stops without knowing where their bus is. Our system shows the exact live location of buses moving on an interactive map, calculates dynamic arrival times (ETA) using the mathematical Haversine formula, displays how crowded each bus is, and updates automatically in real time without refreshing the webpage."*

2. The Problem vs. Our Solution

The Real-World Problem	How Our System Solves It
• Students wait long hours because public/college buses have unpredictable schedules. • Static paper timetables don't show real-time traffic delays or breakdowns. • Students board already overcrowded buses without warning.	• Live bus icon moves smoothly on OpenStreetMap every 2 seconds. • Dynamic ETA countdown (e.g., *"Arriving in 6 mins"*) based on vehicle speed. • Real-time crowd indicator (Low, Moderate, Crowded) showing available seats.

3. The 3 User Roles (Who Uses the System?)

To explain this easily to your teacher, compare it to **Uber**: an Uber app needs a passenger, a driver, and the company manager. Our transit system has the exact same 3 roles:

User Role	Real-World Person	What They Do in the Application
Passenger (Student)	Student or commuter at the bus stop	• **No login needed** (public web page). • Opens the live map to watch their bus moving. • Checks arrival countdown (ETA) & seat availability.
Driver	Bus driver inside the vehicle	• Logs into the onboard tablet (`driver1`). • Sees their assigned route and scheduled stops. • Clicks **"Start Trip"** and **"Finish Trip"**. • Transmits live GPS beacon coordinates.
Admin	College Transport Manager	• Full management portal (`admin`). • Adds/edits buses, routes, drivers, and trips. • Controls the built-in **Java GPS Simulator** (Start, Stop, Speed 1x–5x).

4. Technology Stack (Why Did We Choose These?)

If the teacher asks *"Why did you choose this technology stack?"*, use these exact points:

Technology	Component	Why It Was Used (Teacher Explanation)
Java 17+	Core Language	Mandated by curriculum for strong Object-Oriented Programming (OOP), multi-threading, robustness, and enterprise type safety.
Spring Boot 3.3	Backend Framework	Provides built-in web server (Tomcat), dependency injection, REST APIs, and production-ready security without tedious XML setup.
Thymeleaf + HTML/CSS	Frontend UI	Server-side Java templating. Ensures the application is **100% Java-based** without needing complex JavaScript frameworks like React or Node.js.
Spring WebSocket (STOMP)	Live Updates	Enables bi-directional communication so the server pushes coordinates to the map instantly without requiring the user to reload the page.
MySQL & Spring Data JPA	Database & ORM	Persistent relational storage for buses, routes, stops, and historical location telemetry. H2 database is included for zero-config demos.
Leaflet + OpenStreetMap	Map Engine	100% open-source and free. **Requires zero paid API keys** and no credit card (unlike Google Maps API which is paid).

5. How the System Actually Works (Behind the Scenes)

Here is the technical flow explained simply in 4 easy steps:

Step 1: Coordinate Generation (The GPS Simulator)
Real physical GPS hardware is expensive and impractical inside a classroom. We implemented a multithreaded Java service (GpsSimulatorService.java) that runs every 2 seconds on a background thread. It takes real transit coordinates (Chennai Central Station ➔ Guindy ➔ OMR ➔ Sathyabama Campus) and smoothly moves the simulated bus along that road path.

Step 2: Mathematical Distance & ETA Calculation
Instead of guessing, our Java code uses the mathematical **Great-Circle Haversine Formula**. It calculates the curved spherical distance between the bus's current latitude/longitude and upcoming stops. Dividing distance by current vehicle speed calculates the accurate **Estimated Time of Arrival (ETA)** in minutes.

Step 3: Instant Real-Time Push (WebSocket)
Whenever the bus coordinates change, the server broadcasts a message to the WebSocket channel /topic/bus-locations. Every connected browser receives the new coordinate instantly and glides the bus marker forward on the map without refreshing.

Step 4: Passenger Capacity & Crowding Detection
Each bus tracks capacity and passenger count. When simulated buses arrive at designated stops, passengers realistically board and alight. The system automatically classifies the crowd level as *Low Crowding* (<60%), *Moderate* (60%–85%), or *Crowded* (>85%).

6. Top Viva / Teacher Questions & Plain English Answers

Memorize or refer to these answers during your evaluation:

Q1: Where do the GPS coordinates come from if you don't have physical hardware?

Answer: We built a multithreaded Java GPS simulator. It loads predefined real-world geographic coordinates along actual campus routes. A Java ScheduledExecutorService calculates smooth movement between waypoints and broadcasts periodic updates every 2 seconds. The system also supports real GPS coordinates via driver pings.

Q2: How does the map update without refreshing the page?

Answer: We use Spring WebSocket with STOMP protocol. When the Java server generates a new bus position, it pushes the data over an active WebSocket connection directly to the browser. The lightweight Leaflet JavaScript listener receives the coordinates and moves the bus pin smoothly on the map.

Q3: How do you calculate the arrival time (ETA)?

Answer: In Java, we implemented the Haversine Formula in HaversineDistanceCalculator.java. It calculates the spherical distance in kilometers between the bus's current latitude/longitude and each upcoming stop. Then, using ETA = Distance / Speed, it computes remaining minutes and formats the estimated clock arrival time.

Q4: Why didn't you use React or Angular for the frontend?

Answer: The academic specification strictly required a pure Java-centric enterprise architecture. We used Thymeleaf templates with HTML5 and CSS, which renders cleanly on the server and keeps all business logic, data models, and tracking services strictly inside Java.

Q5: Does this project require paid Google Maps API keys?

Answer: No. Google Maps requires a paid Google Cloud account and billing setup. We integrated Leaflet with OpenStreetMap tiles, which is 100% free, open-source, and never expires. All routing and geodetic math is performed internally by our own Java code, so zero external API keys are required.

Q6: How is security handled?

Answer: We used Spring Security 6. Passwords are encrypted with BCrypt hashing. We implemented Role-Based Access Control (RBAC): the public live map is accessible to all students without login, while administrative fleet management and driver consoles require authenticated credentials.

7. Step-by-Step Live Demonstration Walkthrough

Follow these exact steps when demonstrating the project in front of your teacher:

Step	What to Do	What to Tell the Teacher (Script)
1. Start App	Run command in terminal: java -jar target/realtime-bus-tracking-system-1.0.0.jar --spring.profiles.active=h2	*"The application starts self-contained on port 8080. It automatically initializes database tables and seeds demo buses, routes, and user accounts."*
2. Student View	Open browser to: **http://localhost:8080/track**	*"Here is the commuter map. As you can see, BUS-101 and BUS-102 are moving in real time along the OMR corridor without reloading the page. Clicking a bus shows its speed, next stop, and ETA countdown."*
3. Routes View	Click **Routes** in navbar (http://localhost:8080/routes)	*"Here are the official transit corridors. Clicking 'View Stop Schedule' shows the full vertical timeline of all ordered stops, distances, and scheduled timings."*
4. Admin View	Login at /login with: Username: admin Password: admin123	*"This is the Admin Fleet Command. Admins can add buses, schedule trips, and adjust the GPS simulator speed (e.g., 2x or 5x) in the simulation control room."*
5. Driver View	Login as: Username: driver1 Password: driver123	*"This is the driver's onboard console. The driver can start and finish trips and transmit manual or device GPS sensor pings directly to the server."*

8. Default Demo Credentials Quick Reference

Role	Username	Password	Landing Page
Fleet Administrator	admin	admin123	/admin/dashboard
Transit Driver	driver1	driver123	/driver/dashboard
Student / Commuter	student1	student123	/track (Public)